Sequence Models & NLP – Teaching Machines to Understand Language

Done by:Albatul Altoum

Introduction

This project explores the ability of machines to understand and generate human language using sequence models. Specifically, a character-level LSTM model was implemented to generate text similar in style to Shakespeare. The project demonstrates how recurrent neural networks can learn linguistic patterns, how embeddings help in language representation, and how to evaluate the performance of generative language models using metrics such as perplexity.

1. Code Implementation

The model was implemented using TensorFlow. It is a character-level LSTM that learns to predict the next character in a text sequence. The architecture includes three main layers:
- An Embedding layer: transforms input characters (as integers) into dense vector representations.
- An LSTM layer with 1024 units: processes the sequence and retains long-term dependencies.
- A Dense output layer: projects LSTM outputs to the vocabulary size to predict the next character.

The model was trained for 50 epochs, which is sufficient to demonstrate the behavior of a character-level LSTM. While further training could improve fluency, the goal was to showcase core functionality rather than production-level text.

2. Experimental Setup

The dataset used was Shakespeare's complete works, provided by TensorFlow datasets. Preprocessing included:
- Reading the text and creating a vocabulary of unique characters.
- Mapping each character to a unique integer ID.
- Creating input-target pairs with a fixed sequence length.
- Batching and shuffling the dataset using TensorFlow's data pipeline.
- Using an embedding layer to provide semantic context to the model before feeding into the LSTM.

Padding was implicitly handled by setting a fixed sequence length and using 'drop_remainder=True' in batching.

3. Analysis & Interpretation

Model performance was assessed using the loss value and perplexity metric. The final loss was used to calculate:
- Perplexity = exp(loss)
This reflects how uncertain the model is about its predictions. A lower perplexity means better performance.

In addition, the model was tested with different sequence lengths to observe their effect:
- Shorter sequences train faster but provide less context.
- Longer sequences offer more context but are slower to train and may be prone to vanishing gradients.

The vanishing gradient issue was also addressed. LSTMs are designed to alleviate this issue using gating mechanisms that retain relevant information across many time steps.

4. Explanation

LSTMs improve upon traditional RNNs by addressing the vanishing gradient problem. They do this through:
- Input gates: control how much new information enters the cell.
- Forget gates: determine what information is discarded.
- Output gates: decide what information is output to the next time step.

These components allow LSTMs to maintain a memory over long sequences. Attention mechanisms can enhance this by allowing the model to focus on the most relevant parts of a sequence during generation. Although not implemented in the core model, this concept was explored in the innovation section.

5. Innovation & Insights

To go beyond the basic LSTM implementation, we enhanced the model with pre-trained GloVe embeddings. This improves the quality of the learned representations by providing the model with vectors that already capture semantic relationships between words.

While our model was trained on characters, this experiment demonstrates awareness of how external knowledge can enrich NLP systems. This step reflects an understanding of embedding fine-tuning and integration of linguistic knowledge in deep learning workflows.

Additionally, GPT-2 was briefly explored as an alternative transformer-based generative model using the Hugging Face Transformers library. Due to environment limitations, this was not fully integrated, but it shows awareness of more advanced architectures.

6.Model Output Samples

Below is an example of text generated by the LSTM model after training for 50 epochs on Shakespeare's dataset. The output begins with a prompt (e.g., 'ROMEO:') and the model continues generating characters based on learned patterns.

```
        return start_string + ''.join(generated_text)
```

```
[27] print(generate_text(model, start_string="ROMEO: "))
```

```
ROMEO: Gomege's a ocke tomy Pr, bo?
TENo t d whendir tosen ndet MN ted wnerdon blam' e heas, llye bll RO:
Her;
Yo ast, o VEN: her t gor's nd, avear htr.
LI pe batus

Can t, s se:
Tive tyompede ichatutlfofethaist, y heer

S:

EOUS:
AMI tou tl frs fout tyo ree tous dis CANONG E:
I his, mory meacus
THAULIO:
P
```

6. References

1. TensorFlow Documentation. Text Generation with an RNN.

2. Hugging Face. Transformers Library Documentation.

3. Shakespeare Dataset (TensorFlow). Shakespeare Text Data for Language Modeling.

4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.